

Machine learning in MedyLink

A plain-English guide to the current disease prediction and grouping features, with terms for your viva.

The project question: Which of ten diseases best matches the measurements and symptoms recorded at a visit?

The system learns from synthetic visits with known primary diseases. It compares four classifiers, then uses the selected model for aggregate estimates. The labels are Dengue, Influenza, Hypertension, Type 2 diabetes, Asthma, Anemia, Gastroenteritis, Hypothyroidism, Osteoarthritis and Malaria.

Simple explanation	ML terminology	Where used
Learn disease categories from examples	Supervised multiclass classification	Disease prediction
Compare four learning methods	Logistic Regression, Decision Tree, Random Forest and KNN	Saved model comparison
Group similar measurements	K-Means clustering	Aggregate patient profiles
Find nearby recorded case groups	DBSCAN clustering	Station-area concentrations
Display many measurements on two axes	Principal Component Analysis	Aggregate group centres
Test with different patients	Patient-level holdout and stratified cross-validation	Model evaluation

Current project scope

The future blood-pressure prediction task and its training have been removed. Recorded BP readings remain in clinical reports and charts and can still be inputs to disease classification and measurement grouping. The health score and adherence are arithmetic calculations, not trained prediction models.

Only administrators can access ML insights. Patient and provider access rules remain enforced. Predictions support decisions and are not a diagnosis.

How the data flows

Step	What happens
1. Read visits	Use authorized synthetic observations with a supported primary disease. Keep names, contact details, passwords and doctor identities out of the model.
2. Build inputs	Use age, gender, station, visit month, vitals, blood tests and eight symptom flags. The primary disease is the answer to learn, never a feature.
3. Check timing	Reject values unavailable at the visit. Missing inputs remain missing until fold-specific preprocessing. Diagnosis text and prescriptions are not features.
4. Separate people	Reserve 20% of patients for the final test. Keep every visit from one patient together. Use five shared stratified CV folds within the other 80%.
5. Compare and check	Choose the model with highest average CV macro-F1. Inspect its held-out results only after selection. Enable estimates only if the fixed checks pass.
6. Display aggregates	Use each selected patient's latest eligible record. Return disease counts and group summaries, with counts of one to four hidden.

Filters and saved results

All history includes all recorded dates. Condition search supports partial names and multiple selections; date, line and station filters narrow the view. Counts, charts, groups and estimates reflect the applied cohort. Saved test scores and feature bars keep their training selection until retraining.

A recorded Influenza filter selects visits labelled Influenza. The model may estimate another disease for some of those visits. Those are model disagreements within the selected records, not additional confirmed cases.

An older date selection is a retrospective view with the saved model, not an out-of-time backtest. The model may have been trained with later records.

The four disease algorithms

Algorithm	Simple explanation	Technical explanation
Logistic Regression	Finds a scoring pattern across measurements and symptoms.	A multiclass linear classifier with regularization; a useful comparatively simple baseline.
Decision Tree	Asks a sequence of learned questions.	Partitions feature space into branches. Depth and minimum leaf size limit overfitting.
Random Forest	Lets many different trees vote together.	Uses bootstrap samples and randomized features to reduce dependence on one tree.
KNN	Looks at the most similar past examples.	A distance-based classifier. Scaling keeps large-unit measurements from dominating distance.

Fair preprocessing

Imputation handles missing inputs using only training data. Scaling places numeric features on comparable scales. Encoding represents categories numerically. A scikit-learn Pipeline fits preprocessing inside each fold to avoid leaking validation or test information.

What the metrics mean

Metric	Meaning
Accuracy	The share of disease predictions that were correct overall.
Recall for a disease	Of visits that actually had that primary disease, how many the model found.
Macro-F1	Average disease-specific F1, balancing precision and recall across all ten classes. This selects the winner in CV.
Confusion matrix	Rows are recorded diseases; columns are predicted diseases. Off-diagonal cells are disagreements.

Percentage metrics give each patient equal total weight. Confusion-matrix cells count raw visits, so they need not reproduce the weighted percentages. The held-out test never chooses the winning model.

The measured disease results

Saved run: 2026-10-04T08:45:24.773709+00:00

The report uses **3,892 patients** and **10,997 eligible visits**. Development: 3,113 patients. Final test: 779 patients. No patient appears in both sets.

Method	CV macro-F1	Test accuracy	Test macro-F1
Logistic Regression	74.6%	76.8%	74.8%
Decision Tree	66.7%	66.7%	67.8%
Random Forest	77.9%	79.2%	77.7%
KNN	72.8%	74.5%	72.9%

Selected model: Random Forest. The highest mean CV macro-F1 chose the winner; final-test accuracy did not choose it.

Aggregate predictions require at least **75% test accuracy** and **70% test macro-F1**. The saved Random Forest result passes these simulation thresholds. It is not clinical validation.

What helped the selected model

Input	Test macro-F1 decrease when shuffled
Blood sugar	11.13 points
Hemoglobin	8.81 points
Temperature	6.73 points
Vomiting	5.08 points
Blood oxygen level	3.96 points

Permutation importance shuffles one input and measures the decrease in test macro-F1. A larger drop suggests dependence by this saved model. It is not a disease probability or evidence of medical causation. Correlated features can share information.

Training and evaluation come from a simulated population with deliberately learnable patterns, overlap, label noise and missing values. The numbers describe this experiment, not accuracy in a hospital.

Grouping and using the dashboard

Topic	What the administrator sees
K-Means	Groups with similar standardized measurements; group counts and summaries, without individual members.
PCA	Aggregate group centres on two summary axes. These axes are not BP units, disease severity or a health score.
DBSCAN	Concentrations of nearby synthetic recorded cases. A group is not proof of an outbreak or transmission.

Retrain disease models

Apply your date, condition and area filters, then use **Retrain disease models**. The server repeats the four-model comparison and saves a new report and private model bundle. It uses the selected records; it does not alter clinical data. A narrow selection without adequate examples of all ten diseases keeps the previous successful model.

Run from the project root

```
npm run synthetic:manage -- run_ml_training --dry-run
npm run synthetic:manage -- run_ml_training --sync
```

Disease is the only supported training task and the default. The synthetic database must be configured first. Optional dates: --date-from YYYY-MM-DD --date-to YYYY-MM-DD. Failed runs preserve the previous model; training uses a local background worker.

Viva answers

Question	Answer
Why four models?	Different decision rules are compared fairly on identical data and patient splits.
Why keep patients separate?	Repeated visits share traits; mixing a person across train and test exaggerates generalization.
Why use macro-F1?	It gives each disease equal importance when class counts differ.
Can it diagnose a patient?	No. It is an aggregate synthetic demonstration without independent clinical validation.

Sources: ml/disease_pipeline.py; ml/pipeline.py; ml/reports/disease_evaluation.json. Setup and viva notes: docs/ML_INSIGHTS.md and ml/DISEASE_TASK.md.